

Kombinacija nadgledanog učenja i učenja potkrepljivanjem u treniranju transformera na jednostavnim zadacima

Nikola Vasić

1 Teorijski osvrt

Autoregresivni (dekoder) transformer model na svakom koraku daje verovatnoće mogućih narednih tokena. Prilikom obučavanja menjamo njegove parametre tako da poželjni odgovori postanu verovatniji. Nadgledano učenje (SFT) i učenje potkrepljivanjem (RL) razlikuju se pre svega po informaciji koju koristimo da odredimo šta je poželjno.

Kod SFT-a imamo primer odgovora koji želimo da model nauči da proizvodi. Kod RL-a model generiše odgovor, a zatim dobija ocenu tog odgovora. Arhitektura modela može ostati ista; menja se način formiranja signala za učenje.

1.1 Generisanje teksta kao niz odluka

Neka je x prompt, a $y_1, y_2, \dots$ tokeni odgovora. Model bira naredni token na osnovu prompta i svega što je do tada generisao:

$$\text{prompt} \xrightarrow{y_1} (\text{prompt}, y_1) \xrightarrow{y_2} (\text{prompt}, y_1, y_2) \longrightarrow \dots$$

Ovaj proces možemo opisati standardnim RL pojmovima:

RL pojam	Značenje pri generisanju odgovora
Stanje	Prompt i do tada generisani tokeni
Akcija	Izbor narednog tokena
Polisa	Distribucija verovatnoća narednog tokena koju daje model
Prelaz	Dodavanje izabranog tokena kontekstu
Epizoda	Generisanje jednog odgovora, do stop tokena ili zadate dužine
Nagrada	Ocena generisanog odgovora ili pojedinih koraka

Ovde razmatramo običan odgovor bez pozivanja alata: prelaz samo dopisuje token, dok funkcija za ocenjivanje daje nagradu. Ne mora postojati igra ili fizički svet da bismo imali RL problem. U složenijem agentskom zadatku i reakcije spoljnog okruženja mogu postati deo konteksta.

Oznaka za polisu je:

$$\pi_{\theta}(y_t \mid x, y_{<t}).$$

To je verovatnoća tokena y_t pri datom prefiksu; θ su parametri modela. Kod uzorkovanja token biramo prema toj distribuciji, pa isti prompt može dati različite odgovore. Time model istražuje različite mogućnosti (*exploration*).

1.2 SFT: direktan signal za naredni token

Supervised fine-tuning (SFT) koristi parove promptova i referentnih odgovora. Za svaku poziciju znamo koji token treba predvideti. Prilikom treninga prethodni tokeni odgovora dolaze iz reference (*teacher forcing*).

Uzmimo prompt $87 + 96 =$ i referentni odgovor 183 . Radi preglednosti, u ovom primeru svaku cifru tretiramo kao poseban token; stvarni LLM tokenizator ne mora tako da deli brojeve.

Kontekst tokom SFT-a	Token čiju verovatnoću podstičemo
$87 + 96 =$	1
$87 + 96 = 1$	8
$87 + 96 = 18$	3

Čak i ako bi model sam izabrao 7 na drugoj poziciji, na trećoj poziciji tokom SFT-a dobija referentni prefiks 18. Nije potrebno prvo generisati ceo odgovor da bismo saznali šta da učimo.

Za jedan primer loss možemo zapisati kao:

$$L_{\text{SFT}} = - \sum_t \log \pi_{\theta}(y_t^* \mid x, y_{<t}^*),$$

gde zvezdica označava referentne tokene. Minimizovanje ovog loss-a podstiče veću verovatnoću reference. U praksi se često koristi prosek po tokenima i primerima. Referenca ne mora biti jedini prihvatljiv odgovor, naročito kod otvorenih jezičkih zadataka.

1.3 RL: signal kroz ocenu sopstvenog odgovora

Kod RL-a model prvo uzorkuje odgovor iz trenutne polise. Pretpostavimo da je za isti prompt izabrao 172. Tada su stvarno posećeni konteksti:

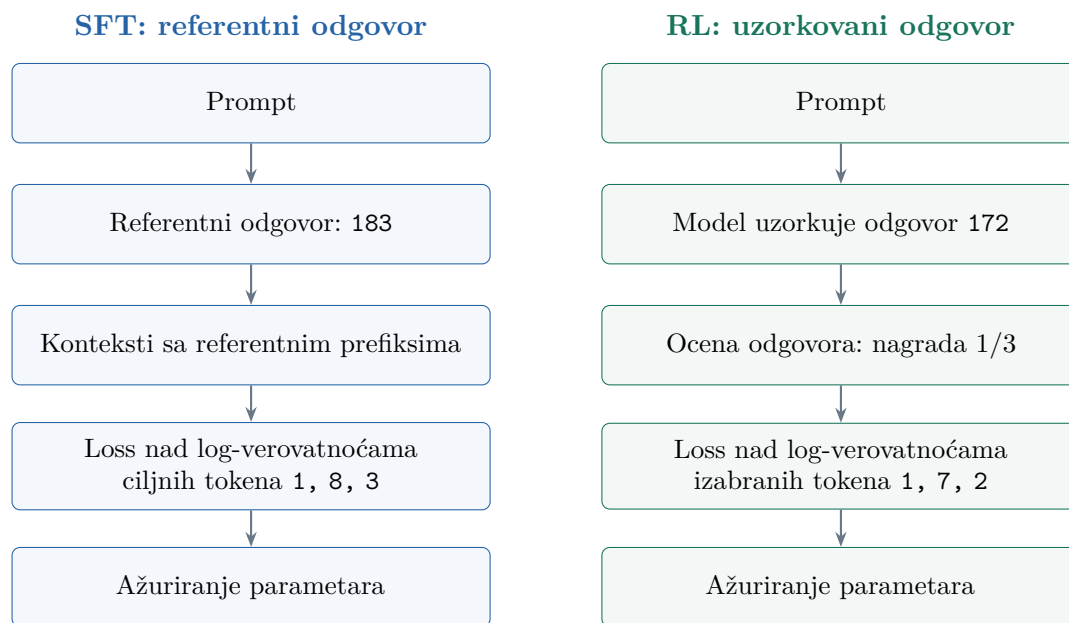
Kontekst tokom generisanja	Izabrani token
$87 + 96 =$	1
$87 + 96 = 1$	7
$87 + 96 = 17$	2

Funkcija nagrade sada ocenjuje **taj odgovor**. Na primer, ako je nagrada udeo tačnih cifara na odgovarajućim pozicijama, odgovor 172 dobija $1/3$ jer se samo prva cifra poklapa sa 183. Exact-match nagrada bi mu dala nulu.

Nagrada može doći iz programske provere, ljudske ocene ili zasebnog modela. **Ne mora biti diferencijabilna**. Tačan odgovor može biti dostupan funkciji nagrade, kao u našem aritmetičkom primeru, ali RL loss ne koristi njegove tokene kao direktne ciljne oznake.

Povratni signal kaže koliko je izabrani odgovor bio dobar; sam po sebi ne kaže koji je token trebalo izabrati umesto 7 ili 2.

Dva toka nadgledanosti



Slika 1: Dva toka supervizije: ciljna sekvenca i ocena uzorkovane sekvence.

	SFT	RL u ovom primeru
Odakle dolazi odgovor za trening?	Iz reference	Model ga uzorkuje
Koji prefiksi se koriste?	Referentni	Uzorkovani
Koji tokeni ulaze u loss?	Ciljni 1, 8, 3	Izabrani 1, 7, 2
Kakav signal dobijamo?	Ciljni token na svakoj poziciji	Ocenu odgovora

1.4 Kako nagrada menja model?

Jedan jednostavan način je **REINFORCE**, algoritam koji koristi nagradu da proceni gradijent očekivane nagrade.

U varijanti sa jednom nagradom na kraju odgovora koristimo:

$$A = R - b, \quad L_{RL} = -A \sum_t \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

R je nagrada, b je referentni nivo (*baseline*), a A prednost (*advantage*). Baseline služi smanjenju varijanse procene gradijenta. Jednostavan izbor je pokretni prosek nagrada prethodnih batch-eva.

- Ako je $A > 0$, update podstiče veću verovatnoću uzorkovanog odgovora.

- Ako je $A < 0$, podstiče manju verovatnoću tog odgovora.
- Ako je $A = 0$, ovaj primer ne daje policy-gradient signal.

Za nagradu $1/3$ i baseline $1/2$, prednost je $-1/6$. Međutim, ista nagrada uz niži baseline može dati pozitivnu prednost. **Netačan odgovor zato nije automatski odgovor sa negativnim signalom.**

U ovoj formulaciji svi izabrani tokeni dele isti skalarni faktor A ; to ne znači da imaju jednake gradijente. Ne dobijamo objašnjenje koji je token zaslužan za uspeh ili grešku. To je problem *dodele zasluge* (*credit assignment*).

Pri računanju gradijenta uzorkovani tokeni, nagrada i baseline tretiraju se kao fiksni. Gradijent prolazi kroz njihove log-verovatnoće u modelu, ne kroz diskretan izbor tokena ili funkciju nagrade.

1.5 Zašto kombinovati ova dva pristupa?

SFT daje precizan signal kada imamo dobre primere željenog ponašanja. RL je koristan kada je lakše oceniti ishod nego navesti jednu idealnu sekvencu: na primer, proveriti da li generisani program prolazi testove. Više različitih odgovora može ostvariti isti cilj.

Zato je prirodno prvo SFT-om naučiti početno ponašanje, a zatim RL-om prilagođavati model cilju koji izražava nagrada. Primer takvog redosleda, sa demonstracijama i ljudskim preferencijama, opisan je u radu [Training language models to follow instructions with human feedback](#).

RL nije automatski bolji: uz retke nagrade model teško pronalazi koristan signal, a loše definisana nagrada može podsticati neželjene odgovore. SFT, s druge strane, ne optimizuje direktno svaku osobinu koja nam je važna pri slobodnom generisanju.

U *praktičnom nastavku* ove ideje ispitujemo na malom transformeru i zadacima sa ciframa. Tamo postoje tačni odgovori za obe metode, pa možemo uporediti šta se menja kada isti tačan odgovor koristimo kao **direktan cilj za tokene** ili kao osnovu za **ocenu uzorkovanog odgovora**. Model u tom eksperimentu učimo od nule; za razliku od doslovnog fine-tuning-a LLM-a, ne polazimo od pretreniranih težina, ali koristimo isti oblik nadgledanog loss-a.

1.5.1 Primene kombinacije nadgledanog učenja i učenja potkrepljivanjem

Kombinacija nadgledanog učenja i reinforcement learning-a ima važnu ulogu u obučavanju savremenih jezičkih modela. Jedan čest obrazac je:

$$\text{pretraining} \longrightarrow \text{SFT} \longrightarrow \text{RL post-training}.$$

Tokom pretraining-a model uči predviđanje narednog tokena na velikoj količini podataka. Ova faza se obično naziva samonadgledanim učenjem, jer ciljni tokeni dolaze iz samog teksta. SFT zatim koristi odabrane primere poželjnih odgovora, dok RL prilagođava ponašanje modela na osnovu nagrade. Ovo je koristan pregled, ali ne i univerzalan recept: konkretni sistemi mogu izostavljati, ponavljati ili kombinovati faze.

Jedna od primena koja je popularizovala RL post-training jeste *reinforcement learning from human feedback* (RLHF). U pristupu prikazanom kroz InstructGPT, model se prvo nadgledano obučava na demonstracijama poželjnih odgovora. Ljudi zatim porede generisane odgovore, a iz tih poređenja uči se model nagrade. Jezički model se potom dodatno obučava RL algoritmom, u tom slučaju PPO-om, da dobija veću nagradu.

Ovde cilj nije samo predviđanje teksta iz skupa podataka, već bolje praćenje instrukcija i stvaranje odgovora koje ljudi ocenjuju kao korisnije. Ljudska procena tako postaje signal za optimizaciju, posredno, kroz naučeni model nagrade.

1.5.2 RL za reasoning i proverljive ishode

Kasnije, veliki značaj RL-a postao je vidljiv i kod *reasoning modela*, posebno na matematičkim i programerskim zadacima. U tim oblastima često možemo automatski proveriti konačan rezultat: da li je broj tačan ili da li program prolazi testove. Takav signal se često naziva *reinforcement learning with verifiable rewards* (RLVR).

Ne moramo unapred označiti svaki korak idealnog rešenja. Model generiše pokušaje, a nagrada podstiče one koji dovode do uspešnog ishoda. Time RL može podsticati korisne strategije rešavanja, proveravanja i ispravljavanja grešaka. OpenAI o1, predstavljen 2024. godine, jedan je od istaknutih primera primene RL-a za poboljšanje ovakvih sposobnosti.

GRPO (*Group Relative Policy Optimization*), uveden u radu DeepSeekMath 2024. godine, postao je posebno uticajan u ovom pravcu. Njegova osnovna ideja je da se za isti prompt uzorkuje više odgovora i njihov kvalitet procenjuje relativno u odnosu na nagrade unutar te grupe. Tako se izbegava potreba za zasebnim naučenim value modelom koji se često koristi uz PPO.

DeepSeek-R1 je 2025. godine dodatno popularizovao ovaj pristup. Posebno je zanimljiv eksperiment R1-Zero: RL je primenjen direktno na **pretrenirani** bazni model, bez prethodne SFT faze. Ovo je pokazatelj da RL može razvijati reasoning ponašanje polazeći od sposobnosti stečenih pretraining-om. Glavni R1 model koristio je složeniji postupak koji kombinuje nadgledane i RL faze.

Napredak reasoning modela zato ne treba pripisati samo jednom algoritmu. Važna je kombinacija sposobnog početnog modela, odgovarajućih zadataka, kvalitetnog signala nagrade i dovoljnog obima treninga. GRPO je jedan važan deo tog razvoja.

Za naš mali eksperiment ovo daje jasnu motivaciju: ispitujemo kako se učenje menja kada model dobija direktne ciljne tokene, kada uči iz ocene sopstvenih odgovora i kada ta dva pristupa koristimo uzastopno, na zadacima i modelima koji su mnogo manji od ovih spomenutih.

2 Projekat

Cilj ovog projekta je implementacija dva načina obučavanja istog malog autoregresivnog transformera: direktno učenje tačnih izlaznih tokena (**SFT**) i učenje iz nagrade za odgovore koje model sam generiše (**REINFORCE**). Treća varijanta prvo koristi SFT, a zatim nastavlja REINFORCE-om, bez ponovne inicijalizacije modela.

Zadaci su namerno jednostavni: za svaki ulaz možemo programski da izračunamo tačan odgovor. Nema spoljnog modela za ocenjivanje, pretreniranih težina ni tekstualnog tokenizatora. Vokabular čini samo deset cifara, sa ID-jevima 0–9.

Važno: Naziv SFT ovde koristimo za običan nadgledani next-token-prediction loss, iako mali model učimo "od nule".

2.1 Zadaci i podaci

Zadatak	Ulaz	Izlaz	Ulazna dužina	Izlazna dužina
Sabiranje modulo 113	105069	061	6	3
Obrtanje cifara	858321	123858	6	6
Ponavljanje cifara	146322	114466332222	6	12

Kod sabiranja, ulaz se deli na dva trocifrena broja: $(105 + 69) \bmod 113 = 61$. Odgovor se dopunjava nulama sleva. Kod ponavljanja se ponavlja **svaka cifra dvaput**, a ne ceo string. Početne nule su

regularni tokeni u sva tri zadatka.

Za modul m postoji m^2 uređenih parova operanada. Podrazumevano koristimo sve parove; `task.num_examples` može da ograniči skup na ravnomeran uzorak jedinstvenih ulaza. Za obrtanje i ponavljanje, od milion mogućih šestocifrenih stringova podrazumevano uzorkujemo 10.000 različitih ulaza, bez konstruisanja celog miliona u memoriji.

`task.dataset_seed` određuje uzorak, `data.split_seed` određuje podelu skupa, a `seed` određuje inicijalizaciju modela, uzorkovanje odgovora i redosled trening primera. Isti uzorak i isti split seed daju iste trening i test primere za sve metode.

Test deo je poslednjih približno 20% jedne fiksne permutacije skupa. Procenat se odnosi na konstruisan skup, ne na sve matematički moguće ulaze. Kod različitih modula menjaju se i zadatak i broj primera.

2.2 Model i poravnanje tokena

Model ima embedding za cifre i naučeni pozicioni embedding, kauzalnu pažnju, MLP blokove i izlaz od deset logita po poziciji. Podrazumevane dimenzije su dva bloka, širina 128 i četiri attention glave. Nema dropout-a ni stop tokena. Broj izlaznih tokena poznat je iz zadatka, pa generisanje traje tačno toliko koraka.

Za batch veličine B , prompt dužine P i odgovor dužine T :

Tensor	Dimenzije
Prompt	$B \times P$
Odgovor	$B \times T$
Spojeni niz bez poslednje cifre	$B \times (P + T - 1)$
Svi logiti	$B \times (P + T - 1) \times 10$
Logiti za odgovor	$B \times T \times 10$

Poslednja pozicija prompta predviđa prvu cifru odgovora. Zato se logiti za odgovor biraju od pozicije $P - 1$, uz indeksiranje od nule. Za podrazumevane zadatke forward sekvence imaju 8, 11 i 17 tokena, a loss obuhvata 3, 6 i 12 predikcija odgovora.

2.3 SFT

SFT koristi tačan odgovor iz skupa. Model pri predviđanju naredne cifre vidi prethodne **tačne** cifre odgovora; (teacher forcing). Kauzalna maska sprečava gledanje budućih cifara. Minimizujemo prosečan negativni logaritam verovatnoće tačnih tokena, odnosno cross-entropy samo na odgovoru:

$$L_{\text{SFT}} = -\frac{1}{BT} \sum_{i=1}^B \sum_{t=1}^T \log p_{\theta}(y_{i,t}^* \mid x_i, y_{i,<t}^*).$$

2.4 REINFORCE

1. Za svaki prompt model uzorkuje jedan odgovor, bez praćenja gradijenata.
2. Funkcija nagrade poredi taj odgovor sa tačnim odgovorom iz skupa.
3. Novim forward prolazom izračunavamo log-verovatnoće uzorkovanih tokena.

4. Množimo zbir tih log-verovatnoća prednošću odgovora u odnosu na baseline.

$$A_i = R_i - b, \quad L_{\text{RL}} = -\frac{1}{B} \sum_{i=1}^B A_i \sum_{t=1}^T \log p_{\theta}(\hat{y}_{i,t} \mid x_i, \hat{y}_{i,<t}).$$

$\hat{y}$ označava uzorkovani odgovor, R_i njegovu nagradu, b baseline, a A_i prednost. Pozitivna prednost podstiče povećanje verovatnoće uzorkovanog odgovora; negativna podstiče smanjenje. Nagrada i prednost nemaju gradijente. Za razliku od SFT-a, tačan odgovor se koristi za ocenu, a ne kao prefiks u RL loss-u.

Baseline počinje od nule, a onda je eksponencijalni pokretni(?) prosek nagrada **prethodnih batch-eva**. Tek nakon računanja trenutnog loss-a ažurira se:

$$b_{\text{novi}} = \beta b_{\text{stari}} + (1 - \beta) \bar{R}_{\text{batch}}, \quad \beta = 0.9 \text{ podrazumevano.}$$

Klasa **Reinforce** čuva baseline između poziva i vraća loss i metriku treninga. Trenutno koristimo jedan rollout po promptu, temperaturu 1 i uzorkovanje bez top-k/top-p filtriranja. Pored osnovnog REINFORCE loss-a, implementirani su opcioni entropijski bonus i KL regularizacija.

2.4.1 Entropijski bonus i KL regularizacija

Za prefiks $s_{i,t} = (x_i, \hat{y}_{i,<t})$, model daje raspodelu verovatnoća nad deset mogućih narednih cifara. Entropija te raspodele je:

$$H_{i,t} = - \sum_{v=0}^9 p_{\theta}(v \mid s_{i,t}) \log p_{\theta}(v \mid s_{i,t}).$$

Entropijski bonus podstiče model da zadrži raznovrsnost pri uzorkovanju, umesto da prerano postane gotovo siguran u jedan izbor. Pošto minimizujemo loss, ovaj član ulazi sa negativnim znakom. Prevelik koeficijent može otežati učenje pouzdanih, tačnih odgovora.

Kod kombinovanog treninga možemo dodatno ograničiti udaljavanje od modela dobijenog SFT fazom. Neposredno pre početka REINFORCE faze pravimo **zamrznutu kopiju modela nakon SFT-a**, sa raspodelom p_{ref} . Njene težine se ne ažuriraju tokom RL treninga.

Na istim prefiksima uzorkovanih odgovora računamo:

$$K_{i,t} = D_{\text{KL}}(p_{\theta}(\cdot \mid s_{i,t}) \parallel p_{\text{ref}}(\cdot \mid s_{i,t})) = \sum_{v=0}^9 p_{\theta}(v \mid s_{i,t}) \log \frac{p_{\theta}(v \mid s_{i,t})}{p_{\text{ref}}(v \mid s_{i,t})}.$$

Ovaj član kažnjava odstupanje trenutne raspodele od SFT reference. Ne zahteva da model ostane identičan referenci, već uvodi cenu za promenu njegovog ponašanja.

Ukupan loss koji minimizujemo je:

$$L_{\text{ukupno}} = L_{\text{RL}} - \lambda_H \frac{1}{B} \sum_{i=1}^B \sum_{t=1}^T H_{i,t} + \lambda_{\text{KL}} \frac{1}{B} \sum_{i=1}^B \sum_{t=1}^T K_{i,t}.$$

Oba regularizaciona člana računaju se nad svih deset mogućih cifara, na prefiksima koje je model uzorkovao. Sabiramo ih po pozicijama odgovora i usrednjavamo po primerima u batch-u, kao i osnovni REINFORCE loss. Nagrada zadatka i EMA baseline ostaju nepromenjeni.

Koeficijente zadajemo kroz `method.entropy_coef` i `method.kl_coef`. Oba su podrazumevano nula. Entropijski bonus možemo koristiti i pri RL treningu od nule, dok pozitivan KL koeficijent u trenutnoj implementaciji zahteva prethodnu SFT fazu.

Na primer:

```
uv run python train.py method=combo method.entropy_coef=0.01 method.kl_coef=0.01
```

U logovima, `train_policy_loss` označava osnovni REINFORCE loss, a `train_loss` uključuje i regularizacijske članove. `train_entropy` i, kada je KL uključen, `train_kl` predstavljaju prosečne vrednosti po izlaznom tokenu, u natima.

2.4.2 Nagrade

Nagrada	Značenje	Zadaci
<code>exact_match</code>	1 ako je ceo odgovor tačan, inače 0	Sva tri
<code>correct_digits</code>	Udeo tačnih cifara na odgovarajućim pozicijama	Sva tri
<code>circular_distance</code>	Normalizovana kružna blizina tačnom ostatku	Sabiranje

Za validan predviđeni broj $\hat{n}$ i tačan broj n koristimo:

$$d = \min(|\hat{n} - n|, m - |\hat{n} - n|), \quad R = 1 - \frac{d}{\lfloor m/2 \rfloor}.$$

Generisani broj van opsega $[0, m - 1]$ dobija nulu; ne svodimo ga automatski modulo m . Maksimalnu nagradu dobija samo kanonski tačan odgovor. Sve nagrade su u intervalu $[0, 1]$.

Exact-match nagrada je retka na početku: pri uniformnom izboru cifara šansa za potpuno tačan odgovor dužine T je 10^{-T} . Duži izlaz zato može znatno otežati RL od nule, iako je samo pravilo zadatka jednostavno.

2.5 Upravljanje zavisnostima i konfiguracijom

Za upravljanje Python okruženjem i paketima koristimo **uv**. Zavisnosti projekta navedene su u datoteci `pyproject.toml`. Komanda `uv sync` priprema okruženje i instalira zavisnosti, dok `uv run` pokreće program unutar tog okruženja:

```
uv sync
uv run python train.py
```

Za upravljanje konfiguracijom koristimo **Hydra**. Podešavanja čuvamo u YAML datotekama, odvojeno za zajedničke parametre, zadatke i metode treninga. Hydra ih spaja u jednu konfiguraciju, a pojedinačne vrednosti možemo promeniti iz komandne linije:

```
uv run python train.py task=reversal method=combo
uv run python train.py method=sft training.epochs=5
```

```
uv sync
uv run python train.py task=reversal method=sft
uv run python train.py task=repetition method=reinforce method.reward=correct_digits
uv run python train.py task=modular_addition method=combo method.phases.sft.epochs=15 method.
    phases.reinforce.epochs=5
```

Kod jedne metode broj epoha određuje `training.epochs`. Kod combo treninga broj epoha određuju `method.phases.sft.epochs` i `method.phases.reinforce.epochs`. Runner po završetku pravi JSON sa konfiguracijama i metrikama svih epoha.

3 Rezultati

3.1 Detalji o implementaciji

`test_loss` je teacher-forced cross-entropy, dok se `test_exact_match` i `test_digit_accuracy` mere greedy generisanjem bez tačnih prefiksa odgovora. `train_reward` dolazi od uzorkovanih trening odgovora, a `test_reward` od greedy test odgovora.

`config.json`, `metrics.jsonl`, `phase_*.pt` i `model.pt` čuvaju se po run-u. Checkpoint sadrži ime zadatka, konfiguracije i težine. Za tačno nastavljjanje prekinutog treninga nedostaju optimizer i RNG stanja; to je budući posao.

Konfiguracija `method=combo` sadrži dve uređene faze. Podaci, težine modela i generator redosleda batch-eva ostaju isti, dok svaka faza dobija nov AdamW. REINFORCE faza počinje baseline-om nula. Čuvamo model nakon obe faze.

3.2 Početni rezultati na modularnom sabiranju

Za sabiranje modulo 113 koristili smo 10.215 trening i 2.554 test primera. Sve rezultate merimo greedy generisanjem na istom test skupu, sa seed-om 42, bez entropijskog bonusa i KL regularizacije.

3.2.1 Šta može REINFORCE od nule?

Prvo smo probali da model učimo samo REINFORCE-om, uz dve različite nagrade. U tabeli su rezultati nakon 10 i 20 epoha istog treninga.

Nagrada	Epohe	Tačni odgovori	Tačne cifre	Test nagrada
<code>correct_digits</code>	10	0.82%	38.78%	0.3878
<code>correct_digits</code>	20	0.82%	38.78%	0.3878
<code>circular_distance</code>	10	2.19%	42.77%	0.7610
<code>circular_distance</code>	20	2.19%	42.63%	0.7612

Kod nagrade `correct_digits`, model se brzo zaglavi: rezultati nakon 10 i 20 epoha potpuno su isti. Kada smo pogledali njegove odgovore, videli smo da za svaki test ulaz generiše 000. Dakle, dolazi do kolapsa polise: model stalno daje isti odgovor, bez obzira na ulaz. Naizgled solidnih 38.78% tačnih cifara zapravo dolazi od toga što se nule često pojavljuju u trocifrenom zapisu ostataka modulo 113.

Sa kružnom nagradom model ipak nauči nešto više. Prosečna test nagrada stiže do približno 0.761 (nasumičnim biranjem dobili bismo nagradu manju od 0.5), mada je samo 2.19% odgovora potpuno tačno. Ovde je bitno da biramo validan broj, nasumično biranje tri cifre često bi dalo broj van opsega, a takav odgovor dobija nagradu nula. Rezultat 0.761 nam zato govori da model uspeva da daje numerički bliže odgovore od ovog slučajnog izbora. Ipak, to je daleko od pouzdanog sabiranja. Ni dodatnih 10 epoha ne pomaže mnogo: broj potpuno tačnih odgovora ostaje isti, a nagrada neprimetno raste sa 0.7610 na 0.7612.

3.2.2 Šta ako prvo uradimo SFT?

Sledeći korak bio je da model prvo učimo 15 epoha SFT-om, pa zatim nastavimo još 15 epoha REINFORCE-om.

Režim	Tačni odgovori	Tačne cifre
SFT, 15 epoha	81.13%	90.68%
SFT 15 + RL 15, <code>correct_digits</code>	84.18%	92.52%
SFT 15 + RL 15, <code>circular_distance</code>	52.58%	74.31%

Već nakon 15 SFT epoha model daje 81.13% potpuno tačnih odgovora. Kada nastavimo REINFORCE-om uz `correct_digits`, taj procenat raste na 84.18%, odnosno za 3.05 procentnih poena. Dakle, ovde RL uspeva da dodatno popravi model koji je već naučio zadatak. Ipak, ako istih dodatnih 15 epoha potrošimo na SFT, dobijamo još bolji rezultat: 95.14%.

Kružna nagrada daje zanimljiviji obrt. Prosečna test nagrada raste sa 0.9683 nakon SFT-a na 0.9763 nakon REINFORCE-a, ali procenat potpuno tačnih odgovora pada sa 81.13% na 52.58%. Model u proseku daje odgovore koji su bliži tačnom broju prema kružnoj udaljenosti, ali ređe pogodi baš taj broj. Ovo lepo pokazuje koliko je važno šta nagrađujemo: bolja nagrada ne mora da znači i više potpuno tačnih odgovora.

Za sada vidimo da SFT samostalno radi veoma dobro, dok REINFORCE od nule slabo uči. S druge strane, nakon SFT-a, REINFORCE ima mnogo bolju početnu osnovu i može da poboljša kriterijum koji mu zadamo.

3.2.3 Kako se rezultat menja tokom RL treninga?

Zatim smo skratili početni SFT trening sa 15 na 10 epoha i pratili šta se dešava tokom narednih 20 REINFORCE epoha. Za ovu proveru koristili smo nagradu `correct_digits`. Rezultate posmatramo nakon svakih pet RL epoha.

Režim	Tačni odgovori	Tačne cifre
SFT 10	40.56%	72.54%
SFT 10 + RL 5	43.93%	74.61%
SFT 10 + RL 10	57.79%	80.03%
SFT 10 + RL 15	65.19%	83.31%
SFT 10 + RL 20	72.91%	86.67%

Ovde je napredak tokom RL faze mnogo izraženiji. Početni model daje 40.56% potpuno tačnih odgovora, a nakon 20 RL epoha stiže do 72.91%: dobitak je čak 32.34 procentna poena. Na prikazanim presecima rastu i tačnost celog odgovora i udeo tačnih cifara.

To je zanimljiv kontrast u odnosu na prethodni eksperiment. Posle 15 SFT epoha model je već bio na 81.13%, pa je dodatnih 15 RL epoha donelo znatno manji dobitak. Posle 10 SFT epoha model ima slabiju, ali očigledno korisnu početnu osnovu, koju REINFORCE uspeva dosta da unapredi.

Ovi rezultati sugerišu da je za RL važno i koliko model već zna: bez SFT-a smo videli stagnaciju, dok posle kraćeg SFT treninga dobijamo dosta veći napredak. Ipak, veći dobitak ne znači i bolji konačni model. Kombinacija 15+15 sa istom nagradom postiže 84.18% tačnih odgovora, više od 72.91% za kombinaciju 10+20, iako obe ukupno traju 30 epoha.

3.2.4 Da li KL pomaže kod kružne nagrade?

Na postavci 10 SFT + 20 RL epoha zatim smo isprobali `circular_distance`, sa KL koeficijentom 0.2 i bez KL regularizacije. Referenca za KL je zamrznuti model dobijen nakon početnih 10 SFT epoha.

Režim	Tačni odgovori	Tačne cifre	Kružna nagrada
SFT 10	40.56%	72.54%	—
SFT 10 + RL 20, bez KL	32.73%	67.49%	0.9710
SFT 10 + RL 20, KL 0.2	43.97%	74.03%	0.9672

Bez KL-a ponovo vidimo raniji problem: kružna nagrada je visoka, ali model daje manje potpuno tačnih odgovora nego pre RL treninga. Sa KL koeficijentom 0.2, tačnost umesto toga raste sa 40.56% na 43.97%. U odnosu na varijantu bez KL-a, to je poboljšanje od 11.24 procentna poena, uz nešto nižu kružnu nagradu. KL ovde pomaže modelu da tokom optimizacije nagrade sačuva više onoga što je naučio SFT-om.

3.3 Obrtanje i ponavljanje cifara

Prelazimo na preostala dva zadatka: obrtanje i ponavljanje cifara. Koristimo podrazumevani model širine 128, sa dva transformer bloka, na 8.000 trening i 2.000 test primera. Ulazi imaju šest cifara, dok izlazi imaju šest za obrtanje, odnosno dvanaest za ponavljanje. RL koristi nagradu `correct_digits`, bez KL regularizacije i entropijskog bonusa.

3.3.1 REINFORCE od nule

Za razliku od modularnog sabiranja, obrtanje cifara model uspeva potpuno da reši na test skupu već nakon tri epohe samostalnog REINFORCE treninga. Kod ponavljanja rezultat je slabiji:

Zadatak, 3 RL epohe	Tačni odgovori	Tačne cifre
Obrtanje cifara	100.00%	100.00%
Ponavljanje cifara	7.15%	79.00%

Ova razlika pokazuje zašto pratimo obe metrike. Na ponavljanju model već pogađa približno 79% cifara, ali je samo 7.15% celih odgovora tačno. Izlaz ovde ima dvanaest cifara i dovoljno je pogrešiti jednu da ceo odgovor bude ocenjen kao netačan. Duži izlaz zato može doprineti ovoj razlici, mada se zadaci razlikuju i po pravilu koje model treba da nauči.

3.3.2 Jedna SFT epoha kao početna osnova

Za ponavljanje smo zatim probali isti ukupan broj epoha, ali sa jednom SFT epohom na početku i dve REINFORCE epohe nakon nje.

Trenutak treninga	Tačni odgovori	Tačne cifre
Nakon 1 SFT epohe	19.25%	80.72%
Nakon 1 SFT + 1 RL epohe	99.35%	99.95%
Nakon 1 SFT + 2 RL epohe	100.00%	100.00%

Ovde je efekat početnog SFT-a posebno upečatljiv. Posle jedne epohe model još uvek greši na većini celih odgovora, ali već sledeća RL epoha podiže exact-match tačnost sa 19.25% na 99.35%. Nakon druge RL epohe svi test odgovori su tačni.

Dakle, uz isti budžet od tri epohe, samostalni REINFORCE postiže 7.15% potpuno tačnih odgovora, dok kombinacija 1+2 stiže do 100%. Jedna SFT epoha ovde daje početnu osnovu koju RL vrlo brzo uspeva da iskoristi.

Sa ovim modelom zadaci brzo dostižu maksimalnu test tačnost, pa ostaje malo prostora da pratimo razlike između metoda. Zato ćemo u sledećim eksperimentima smanjiti širinu modela i proveriti kako se tada menja tok učenja.

3.4 Šta se dešava sa manjim modelom?

Da bismo otežali prethodne zadatke, smanjili smo `d_model` sa 128 na 16. Time ne smanjujemo samo embedding, već i širinu attention projekcija i MLP slojeva. Sada model za obrtanje ima samo **7.088 parametara**, a model za ponavljanje **7.184**. Prethodni modeli širine 128 imali su približno 400.000 parametara, pa smo broj parametara smanjili oko 56 puta. Oba manja modela i dalje imaju samo dva transformer bloka i četiri attention glave. Sada razlika između samostalnog REINFORCE-a i treninga sa početnom SFT fazom postaje mnogo vidljivija.

U nastavku pratimo rezultate nakon određenog broja epoha, a ne pojedinačnih optimizer koraka.

3.4.1 Obrtanje cifara

Na obrtanju cifara, mali model nakon 20 epoha samostalnog REINFORCE treninga postiže 57.85% tačnih cifara, ali samo 0.60% potpuno tačnih odgovora. Zatim smo isti budžet od 20 epoha raspodelili na 5 SFT i 15 REINFORCE epoha.

Režim	Tačni odgovori	Tačne cifre
Samo RL, 20 epoha	0.60%	57.85%
SFT 5	0.05%	39.61%
SFT 5 + RL 5	1.20%	61.36%
SFT 5 + RL 10	24.90%	83.30%
SFT 5 + RL 12	84.90%	97.44%
SFT 5 + RL 14	99.75%	99.96%
SFT 5 + RL 15	100.00%	100.00%

Posle početnih pet SFT epoha model gotovo da ne daje nijedan potpuno tačan odgovor. Ipak, već pogađa 39.61% cifara, pa početna osnova postoji čak i kada se slabo vidi kroz exact-match metriku.

Tokom RL faze napredak u početku deluje spor: posle pet epoha potpuno je tačno tek 1.20% odgovora. Zatim dolazi do velikog pomaka. Između desete i dvanaeste RL epohe exact-match tačnost raste sa 24.90% na 84.90%, a do petnaeste stiže do 100%.

Ovde početni SFT pravi veliku razliku: uz isti ukupan broj epoha, samostalni RL ostaje na 0.60% tačnih odgovora, dok kombinacija 5+15 rešava ceo test skup.

3.4.2 Ponavljanje cifara

Kod ponavljanja cifara razlika je slična. Nakon 30 epoha samostalnog REINFORCE treninga model pogađa 26.80% cifara, ali nijedan ceo test odgovor. Za kombinovani trening istih 30 epoha delimo na 10 SFT i 20 REINFORCE epoha.

Režim	Tačni odgovori	Tačne cifre
Samo RL, 30 epoha	0.00%	26.80%
SFT 10	1.30%	54.74%
SFT 10 + RL 5	5.90%	71.83%
SFT 10 + RL 10	22.00%	84.51%
SFT 10 + RL 15	80.10%	97.08%
SFT 10 + RL 20	99.05%	99.90%

I ovde početni SFT model retko daje potpuno tačan odgovor, ali već pogađa više od polovine cifara. REINFORCE zatim postepeno podiže obe metrike, sve do 99.05% potpuno tačnih odgovora.

Na oba zadatka vidimo da mali broj potpuno tačnih odgovora nakon SFT-a ne znači da model nije naučio ništa korisno. Delimično znanje, vidljivo kroz tačnost pojedinačnih cifara, u ovim eksperimentima dovoljno je da RL nastavi uspešno učenje.

Sa manjim modelom početna SFT faza zato postaje posebno važna. Samostalni REINFORCE ostaje daleko od rešenja i nakon 20 ili 30 epoha, dok kombinovani trening sa istim ukupnim brojem epoha stiže do potpune ili gotovo potpune test tačnosti.